

The Object Counting Algebra (Discussion)

The vector of this text [

God will exist.

]

Purpose of this text [

Equip the reader with the 'Object-Counting Algebra'.

]

Object Counting Algebra

The Object Counting Algebra is a configuration-based arithmetic in which an object called o is a countable constituent, a finite alphabet supplies the primitive object types, and each admissible configuration receives a canonical sequential identifier.

Addition, subtraction, multiplication, and division operate on the configurations referenced by those identifiers. Their outputs are identifiers for resulting configurations, collections of identifiers for admissible solutions, or explicit indications that an operation

cannot be completed within the chosen configuration domain.

The foundational distinction is between the identifier of a configuration and the configuration itself. An identifier provides a canonical address. A configuration provides the object multiplicities and structural content on which the algebra operates. An interpretation assigns domain-specific meaning to that content.

Objects, alphabet, and configurations

An object instance is an individual occurrence of a primitive object type. The alphabet is a finite, nonempty collection of those primitive types.

A configuration is a finite collection of object instances in which repeated occurrences are permitted. In this construction, configurations are represented as finite multisets, meaning that multiplicity matters but ordering does not.

Two configurations are identical when they contain the same number of instances of every primitive type. Rearranging their instances does not create a different configuration.

A configuration can therefore be represented as a multiplicity vector. Each coordinate corresponds to one primitive type, and its value is a nonnegative integer specifying the number of instances of that type. Equivalently, the configuration is a formal sum of

primitive objects with nonnegative integer coefficients.

For example, a configuration over a three-type alphabet might contain two instances of the first type, one instance of the second type, and no instances of the third type. Its structural content is precisely that distribution of multiplicities.

The total cardinality of a configuration is the sum of its multiplicities. The empty configuration has zero occurrences of every primitive type and total cardinality zero.

A finite alphabet does not imply a finite configuration space. Provided multiplicities are unbounded, configurations can contain arbitrarily many objects. The primitive vocabulary remains finite while the collection of finite configurations is countably infinite.

Canonical configuration identifiers should also be distinguished from instance identifiers. Two distinct instances of the same primitive type contribute a multiplicity of two. Their individual provenance is not preserved by a multiplicity-only representation. Persistent instance identity requires an additional identification layer.

Likewise, ordering, geometry, connectivity, and spatial arrangement are not encoded by multiplicities alone. A configuration model requiring those properties must explicitly include them and define an appropriate canonical representation.

Canonical sequential identification

Every configuration receives exactly one sequential identifier, and every valid identifier refers to exactly one configuration.

To establish this correspondence, first fix an ordering of the primitive alphabet. Then enumerate configurations by increasing total cardinality. Within each cardinality level, use a fixed lexicographic ordering of their multiplicity vectors.

Lexicographic ordering compares the multiplicities in the declared alphabet order. The first coordinate at which two configurations differ determines their relative position.

The empty configuration receives identifier zero. Configurations containing one object follow, then configurations containing two objects, and so forth. Within every cardinality level, the fixed comparison rule determines the sequence.

For a specified total object count, the number of configurations is the binomial coefficient whose upper argument is the object count plus the alphabet size minus one, and whose lower argument is the alphabet size minus one.

Consequently, each cardinality level contains finitely many configurations. Enumerating those finite levels successively produces a complete sequential indexing of

all finite configurations.

The encoder maps a configuration to its sequential identifier. The decoder maps an identifier to its configuration.

Encoding and then decoding must recover the original configuration. Decoding and then encoding must recover the original identifier. These round-trip properties establish a bijective correspondence between the configuration space and the sequential identifier space.

The identifier is therefore a referential token, not a direct measurement of object quantity. Its magnitude does not generally equal the cardinality, complexity, semantic importance, or physical magnitude of its configuration.

The operational principle

Every identifier-level operation follows the same conceptual procedure.

The operands are decoded into configurations. The selected configuration operation is applied. Every valid resulting configuration is expressed in canonical form. The encoder then returns its sequential identifier.

Where the operation admits several solutions, each solution is canonicalized and encoded independently.

This procedure is a transport of algebraic structure through an encoding. The configuration space carries the

substantive operations, while the identifier space provides an equivalent representation of those operations.

Ordinary numerical arithmetic on the identifiers is not substituted for configuration arithmetic. Adding two address numbers does not, by itself, combine the objects stored at those addresses.

A consistent change of identifiers changes the names of configurations without changing their algebraic relationships. This reindexing invariance distinguishes the substance of the algebra from an incidental numbering convention.

Encoding establishes representational consistency. It does not independently establish semantic validity. Semantic validity additionally depends on the configuration definitions, composition rules, interpretation, and applicable constraints.

Addition as aggregation

Addition combines the multiplicities of corresponding primitive types.

For each primitive type, the output multiplicity is the multiplicity in the first configuration plus the multiplicity in the second configuration.

Suppose the first configuration contains two instances of the first type, one instance of the second type, and no instances of the third type. Suppose the second

configuration contains one instance of the first type, no instances of the second type, and three instances of the third type.

Their sum contains three instances of the first type, one instance of the second type, and three instances of the third type.

The resulting configuration is encoded, and its canonical identifier becomes the output.

The semantic interpretations of addition include aggregation, accumulation, collection, inventory combination, and multiset merging.

This is union with multiplicity, rather than ordinary set union. Repeated objects remain countable occurrences; they are not collapsed into a single occurrence merely because they share a type.

Addition is associative and commutative. Regrouping additions or exchanging their operands does not change the resulting multiplicities.

The empty configuration is the additive identity. Adding it leaves a configuration unchanged.

Total cardinality is additive: the number of objects in the sum equals the combined number of objects in its operands.

Under this operation, the configuration space forms a free

commutative additive monoid generated by the primitive alphabet.

Subtraction as exact removal

Subtraction removes the multiplicities specified by one configuration from another configuration.

For every primitive type, the required removal multiplicity is subtracted from the available multiplicity.

Within the nonnegative counting domain, subtraction is admissible only when every requested multiplicity is available. A surplus of one primitive type does not compensate for a shortage of another primitive type unless additional conversion rules explicitly permit that transformation.

Subtracting the second configuration in the preceding addition example from the aggregated configuration recovers the first configuration.

The semantic interpretations of subtraction include removal, consumption, extraction, cancellation, depletion, and residual computation.

Subtracting the empty configuration leaves the original configuration unchanged. Subtracting a configuration from itself produces the empty configuration.

Subtraction is a partial operation on ordinary

configurations. An attempted removal can fail when one or more primitive multiplicities would become negative.

Such failure must be distinguished from the empty configuration. An empty configuration is a valid state containing no objects. A subtraction failure indicates that the requested exact residual is unavailable within the nonnegative domain.

Silently replacing negative multiplicities with zero would define a different operation. That truncated operation would discard deficit information and would not preserve all the inverse relationships of exact subtraction.

Signed configurations and formal deficits

A broader algebra can permit integer multiplicities rather than only nonnegative integer multiplicities.

Positive coefficients represent available quantities. Zero coefficients represent absence. Negative coefficients represent formal deficits, obligations, counterbalances, or additive inverses, according to the chosen interpretation.

A negative multiplicity does not automatically mean that a negative number of physical objects exists. It is a formal algebraic value requiring an explicit semantic interpretation.

In this signed extension, subtraction becomes total. Every configuration can be subtracted from every other

configuration because negative residual coefficients are admitted.

The additive structure becomes a free abelian group, obtained through the additive group completion of the original counting structure.

The identifier system must also be extended. A decoder designed only for nonnegative configurations cannot represent signed configurations without modification.

One suitable signed enumeration orders configurations by the sum of the absolute values of their multiplicities, followed by a fixed ordering within each level. This preserves finite enumeration levels while admitting positive and negative coefficients.

Where the multiplication is derived from a primitive monoid, the nonnegative construction gives a monoid semiring, while the corresponding signed construction gives a monoid ring.

Multiplication as primitive composition

Multiplication requires an explicit composition rule for every ordered pair of primitive types.

That rule specifies the configuration produced when an object of the first type composes with an object of the second type.

A primitive composition may produce one primitive object,

several objects, or the empty configuration, depending on the declared rules.

To multiply two configurations, consider every primitive type present in the first configuration together with every primitive type present in the second configuration.

For each ordered type pair, multiply the two input multiplicities. That product determines how many copies of the pair's prescribed output configuration are contributed.

Finally, aggregate the contributions from all ordered pairs.

Multiplication is therefore a distributive extension of primitive composition to composite configurations.

Its semantic interpretations can include interaction, functional composition, replication, pairwise synthesis, production, transformation, and combinatorial construction.

The semantic choice must be explicit. A production rule, a transformation-composition rule, and a replication rule are different models even when they share this algebraic framework.

Multiplication by an ordinary counting quantity can also be represented internally when a multiplicative identity exists. A scalar configuration containing a specified number of copies of that identity replicates another

configuration by that quantity.

The scalar configuration must not be confused with an identifier having the same numerical value.

Multiplicative laws and the monoid specialization

Distributivity follows from extending primitive composition through multiplicity aggregation. Multiplying a configuration by an aggregate gives the same configuration as multiplying it by each constituent configuration and then aggregating the products.

However, an arbitrary primitive composition table does not automatically provide every conventional multiplication law.

Associativity requires compatible composition rules. Commutativity requires symmetry under exchanging the primitive operands. A multiplicative identity exists only when the composition rules admit a configuration that leaves every configuration unchanged under multiplication on both sides.

A particularly coherent specialization makes the primitive alphabet a finite monoid.

A monoid supplies an associative primitive composition operation and a distinguished identity type. Composing any two primitive types produces exactly one primitive type within the same alphabet.

The configuration containing one instance of the identity type becomes the multiplicative identity.

It is not the empty configuration. Its sequential identifier is not necessarily one.

In this specialization, multiplication is a convolution over primitive composition. For each output type, its multiplicity is obtained by summing the products of input multiplicities over all ordered primitive pairs whose composition produces that output type.

The resulting configuration structure is the monoid semiring with nonnegative integer coefficients.

Addition is associative and commutative. Multiplication is associative. Multiplication distributes over addition. The empty configuration is the additive identity and is absorbing under multiplication. The one-object identity configuration is the multiplicative identity.

Multiplication is commutative when the underlying primitive monoid is commutative.

Because every primitive pair produces exactly one primitive object in this specialization, total cardinality is multiplicative. The product contains as many objects as there are ordered pairs of input occurrences.

For a more general composition table, this cardinality law need not hold. Primitive pairs may produce different numbers of output objects.

Division as inverse composition

Division asks which configurations could have generated a specified configuration through multiplication with a specified divisor.

The dividend is the configuration to be reproduced. The divisor is the configuration held fixed. A quotient is any configuration that satisfies the required composition relationship.

For a right-hand divisor, a candidate quotient is valid when multiplying that candidate by the divisor on its right reproduces the dividend exactly.

Division returns the collection of all valid quotient configurations, subsequently represented by their canonical identifiers.

Its semantic interpretations include factorization, decomposition, reverse composition, generator recovery, source inference, and constituent reconstruction.

Division may have no solution, one solution, several solutions, or potentially infinitely many solutions.

No solution produces an empty collection of quotient identifiers. One solution produces a collection containing one identifier. Several solutions produce a collection containing the identifiers of all admissible quotient configurations.

An empty collection of solutions is not the same as a solution consisting of the empty configuration.

Where multiplication is noncommutative, the side of the divisor matters. Solving for a configuration multiplied by a fixed right-hand divisor is generally different from solving for a configuration multiplied by a fixed left-hand divisor.

Division is therefore naturally a relation or set-valued operation, rather than an automatically single-valued inverse.

Division constraints and exceptional cases

In the finite monoid specialization, every quotient associated with a nonempty divisor must have a total cardinality compatible with the multiplicative cardinality law.

The dividend cardinality must be divisible by the divisor cardinality. When that numerical divisibility fails, no exact configuration quotient exists.

Numerical divisibility is necessary but not sufficient. The primitive-type distribution must also satisfy the composition rules.

For a nonempty divisor, the required quotient cardinality is fixed. Since the alphabet is finite, only finitely many configurations have that cardinality. The exact quotient

collection is therefore finite in this specialization.

The empty divisor produces distinctive cases.

A nonempty configuration cannot be obtained by multiplying anything by the empty configuration, so a nonempty dividend divided by the empty configuration has no solution.

The empty configuration multiplied by any configuration remains empty. Consequently, dividing the empty configuration by itself admits every configuration as a quotient and produces an infinite solution collection.

Dividing the empty configuration by a nonempty divisor has the unique empty quotient in the finite monoid specialization.

More general primitive composition tables can exhibit additional degeneracies, including nonempty configurations that compose to the empty configuration.

A quotient with remainder is not automatically supplied by this construction. Defining one requires additional conventions governing residuals, admissible decompositions, ordering, or optimization.

Semantic interpretation

An interpretation function assigns domain meaning to each configuration.

That meaning might describe an inventory, a resource state, a composite transformation, a production outcome, a computational process, or another explicitly defined semantic object.

The identifier locates the configuration. The configuration specifies its primitive content. The interpretation explains what that content means within the application domain.

Meaningful arithmetic requires compatibility between these layers.

For addition, the interpretation should explain what combining the represented configurations means. For subtraction, it should explain what exact removal means. For multiplication, it should explain primitive composition and its extension to composite configurations. For division, it should explain what recovering a valid generating configuration establishes.

When interpretation preserves the relevant operations, it acts as a homomorphism. Interpreting a configuration-level result then agrees with performing the corresponding semantic operation on the interpreted operands.

Canonical configuration identity and semantic equivalence need not coincide. Two different configurations can have the same interpretation while retaining different canonical identifiers.

Merging such configurations into one algebraic state

requires an explicitly declared equivalence relation that is preserved by the operations. This preservation condition is a congruence requirement. Without it, results could depend on which representative configuration was selected.

A complete illustrative model

Consider an alphabet containing two primitive transformation types, called neutral and toggle.

A neutral object represents leaving a binary condition unchanged. A toggle object represents reversing that condition.

Composing neutral with neutral produces neutral. Composing neutral with toggle produces toggle. Composing toggle with neutral produces toggle. Composing toggle with toggle produces neutral.

These rules form a finite commutative monoid with neutral as its identity.

Declare neutral as the first alphabet type and toggle as the second. Enumerate configurations by increasing cardinality and then by ascending lexicographic multiplicity order.

The empty configuration receives identifier zero. One toggle receives identifier one. One neutral receives identifier two. Two toggles receive identifier three. One neutral together with one toggle receives identifier four.

Two neutrals receive identifier five.

Object addition applied to identifier one and identifier two produces identifier four. The operation combines one toggle with one neutral. Ordinary numerical addition of those identifiers would not produce the correct configuration address.

Object subtraction applied to identifier four and identifier one produces identifier two. Removing one toggle from the mixed configuration leaves one neutral.

Object multiplication applied to identifier one and identifier one produces identifier two. Composing two toggle transformations produces the neutral transformation.

Object division applied to identifier four with identifier four as the right-hand divisor has two quotient configurations.

One quotient contains a single neutral object. Multiplying it by the mixed configuration leaves that configuration unchanged.

The other quotient contains a single toggle object. Multiplication exchanges the neutral and toggle contributions, but the aggregate still contains one of each.

The quotient collection therefore contains identifier one and identifier two.

This demonstrates that semantically meaningful division can be nonunique even when neither operand is empty. The ambiguity follows from the composition structure, not from an encoding failure.

The complete algebraic meaning

The complete construction consists of a finite primitive alphabet, an admissible configuration domain, a canonical sequential enumeration, an encoder, a decoder, a semantic interpretation, aggregation, exact residual subtraction, distributive composition, inverse-composition solution collections, an additive identity, and a multiplicative identity wherever the composition rules provide one.

A failure indicator may be added for operations that are undefined within the selected domain. It must remain distinguishable from valid empty configurations and empty solution collections.

Addition and multiplication are configuration-valued operations under the stated closure conditions. Exact subtraction is partial in the nonnegative domain. Division is collection-valued. These distinctions should remain explicit rather than being concealed by the shared vocabulary of arithmetic.

The construction is therefore not automatically a field, and its four operator names do not imply that all conventional numerical identities apply without qualification.

Its essential invariant is that identifier-level arithmetic faithfully represents configuration-level algebra.

The sequential identifiers form the index space. The configurations form the state space. The primitive alphabet supplies the generative vocabulary. The composition rules determine interaction. The interpretation supplies domain meaning. The operators provide a semantic transformation calculus over the indexed configurations.

Counting determines how many instances occur. Configuration determines which types occur and with what multiplicities. Composition determines what their interactions produce. Division determines which configurations could have produced a specified result. Sequential identification makes each admissible configuration addressable without replacing its meaning with the numerical magnitude of its address.

Question and Answers 1

prompt1

Formally define 'Object counting algebra'.

Object Counting Algebra

Object Counting Algebra is a canonically indexed, semantically interpreted algebra of finite typed configurations in which addition aggregates object multiplicities, subtraction extracts an admissible additive residual, multiplication extends a declared primitive composition law distributively, and division identifies every configuration satisfying an exact multiplicative reconstruction constraint.

Its configuration space contains the mathematical states on which the operations act. Its identifier space provides a sequential address for each configuration. Its interpretation assigns application semantics to those configurations and specifies how the algebraic operations correspond to meaningful transformations.

The following definition formalizes the multiset construction specified in your [engineering codex](#).

Primitive objects and configuration space

An **object occurrence** is one countable instance of a primitive object type. A **primitive alphabet** is a finite, nonempty, explicitly ordered collection of such types.

A **configuration** is a finite multiset of object occurrences drawn from this alphabet. Equivalently, a configuration assigns a nonnegative integer multiplicity to every primitive type. Each multiplicity records how many occurrences of its associated type are present.

For the basic construction, every such multiplicity assignment is admissible. Object counts may increase without a fixed upper bound, while every individual configuration remains finite.

Configuration equality holds precisely when the multiplicities of every corresponding primitive type agree. This equality criterion makes the configuration representation extensional: the complete multiplicity distribution determines the configuration.

A **primitive configuration** contains exactly one occurrence of one primitive type and zero occurrences of every other type. These primitive configurations generate the entire configuration space through finite aggregation.

The **empty configuration** assigns zero multiplicity to every primitive type. It is a valid configuration and supplies the additive identity.

The **cardinality** of a configuration is the sum of all its primitive multiplicities. Cardinality records total occurrence count. The multiplicity distribution additionally records the allocation of that count among primitive types.

This definition treats occurrences of the same primitive type as interchangeable for configuration equality. Persistent instance identity, provenance, sequence, geometry, and connectivity require additional state components whenever an application needs those distinctions.

Although the alphabet is finite, the complete configuration space is countably infinite. Unbounded multiplicities permit arbitrarily large configurations, and the finiteness of the alphabet ensures that only finitely many configurations have any specified cardinality.

Canonical sequential identification

A **canonical sequential identification** is a bijection between the configuration space and the nonnegative integers, established by a fixed enumeration convention.

For the convention in your codex, configurations are ordered first by increasing cardinality. Configurations having the same cardinality are ordered by ascending lexicographic comparison of their multiplicities, using the declared alphabet order.

Lexicographic comparison examines corresponding multiplicities in sequence. The first unequal multiplicities determine the ordering of the two configurations.

The empty configuration receives identifier zero. Every subsequent configuration receives the next sequential identifier in the enumeration.

Because each cardinality level is finite, every configuration occurs at a finite position. Because every finite multiplicity distribution belongs to a cardinality level, the enumeration covers the entire configuration space.

The **encoder** assigns each configuration its identifier. The **decoder** recovers the unique configuration associated with an identifier.

These functions satisfy two inverse conditions. Decoding an encoded configuration recovers that configuration. Encoding a decoded identifier recovers that identifier.

Canonicity is relative to the fixed alphabet order and enumeration convention. Once those choices are fixed, each configuration has exactly one canonical

address.

The numerical magnitude of an identifier records its enumeration position. Object cardinality is recovered by decoding the identifier and summing the resulting multiplicities.

Every operation on identifiers is defined by **transport of structure**: decode the operands, apply the corresponding configuration operation, and encode each resulting configuration. For an operation returning several configurations, encoding acts separately on every member of the solution collection.

A consistent reindexing preserves the algebraic relationships between configurations. The encoder changes their addresses while transporting addition, multiplication, subtraction admissibility, and division solutions through the same bijection.

Addition as multiplicity aggregation

Object addition is the total binary operation that aggregates two configurations by adding their corresponding primitive multiplicities.

For every primitive type, the output contains as many occurrences as the combined number of occurrences of that type in the two operands.

This definition determines addition uniquely and gives it the following properties.

Associativity means that aggregating three configurations produces the same result under either grouping.

Commutativity means that exchanging the two operands preserves the result.

Additive identity means that aggregating the empty configuration with any configuration preserves that configuration.

Cancellation means that two configurations are equal whenever aggregating the same third configuration with each produces equal results.

Together with the empty configuration, the complete configuration space is the **free commutative additive monoid generated by the primitive alphabet**.

Finite multisets are a standard realization of this construction. [Choudhury and Fiore, *Free Commutative Monoids in Homotopy Type Theory*](#)

Its freeness has a precise universal meaning: any assignment of primitive types to elements of another commutative additive monoid extends uniquely to an addition preserving mapping on all configurations. The extension interprets a configuration by aggregating the assigned primitive meanings according to their multiplicities.

Cardinality preserves addition. The cardinality of an aggregate is the sum of the cardinalities of its operands.

Semantically, addition can represent inventory combination, resource accumulation, collection construction, or aggregation of formal alternatives. The interpretation specifies which meaning applies.

Subtraction as exact additive residual

Object subtraction is the partial binary operation that returns the unique configuration whose aggregation with the removal configuration reconstructs the original configuration.

The first operand specifies the available configuration. The second operand specifies the configuration to remove.

A residual exists precisely when, for every primitive type, the requested removal multiplicity is no greater than the available multiplicity. When this condition holds, each residual multiplicity is the corresponding available multiplicity reduced by the corresponding removal multiplicity.

Residual uniqueness follows from additive cancellation.

This definition also establishes a natural **subconfiguration order**. One configuration is a subconfiguration of another precisely when every multiplicity in the first is bounded above by the corresponding multiplicity in the second. Exact subtraction is defined precisely when the removal operand is a subconfiguration of the available operand.

Subtracting the empty configuration preserves the original configuration. Subtracting a configuration from itself returns the empty configuration. Aggregating a residual with the configuration removed from it reconstructs the original operand.

A shortage of one primitive type makes the subtraction undefined, even when other primitive types have surplus multiplicities. Conversion between types requires an explicitly declared transformation rule.

An undefined subtraction has a different status from a valid empty residual. The empty residual satisfies the reconstruction condition. An undefined subtraction indicates that no admissible residual satisfies it.

An implementation may represent undefined subtraction through a separate diagnostic value. That diagnostic belongs to an extended result type and must remain distinguishable from every valid configuration identifier.

Multiplication as distributive primitive composition

Primitive composition is a total rule assigning a finite output configuration to every ordered pair of primitive types.

A primitive pair may produce an empty configuration, a single primitive occurrence, or a configuration containing several occurrences of one or more primitive types.

The complete composition table supplies the substantive interaction rules of the algebra. These rules may express transformation composition, synthesis, replication, production, or another specified interaction semantics.

Object multiplication is the unique extension of primitive composition that preserves addition separately in each operand.

To determine the product of two configurations, consider every ordered pair consisting of a primitive type from the first operand and a primitive type from the second operand.

For each pair, multiply the multiplicity of the first type in the first operand by the multiplicity of the second type in the second operand. This determines how many copies of the pair's prescribed output configuration contribute to the product.

Aggregate all these contributions. The resulting multiplicity distribution is the product configuration.

The multiplicities appearing in the primitive output configurations can equivalently be regarded as **nonnegative integer structure constants**. These constants determine how each ordered pair of input types contributes to every output type.

This construction guarantees **left distributivity** and **right distributivity**. Multiplication by an aggregate agrees with aggregation of the corresponding products, whichever operand contains the aggregate.

It also guarantees that the empty configuration is **multiplicatively absorbing**. Multiplying an empty configuration by any configuration, on either side, produces the empty configuration because there are no contributing pairs of occurrences.

The product is deterministic once the primitive composition table is fixed. Multiple occurrences in its output constitute one configuration with specified multiplicities.

Additional multiplicative axioms

Associativity, commutativity, and a multiplicative identity require additional conditions on primitive composition.

Multiplicative associativity holds when composing three configurations gives the same output under both possible groupings. Because multiplication extends distributively from primitive configurations, associativity holds throughout the

configuration space precisely when it holds for every ordered triple of primitive configurations.

For each primitive triple, multiplying the first two and then multiplying their result by the third must agree with multiplying the last two and then multiplying the first by their result. This is a finite verification condition because the alphabet is finite.

Multiplicative commutativity holds precisely when every primitive pair has the same prescribed output as the reversed pair.

A **multiplicative identity** is a configuration whose multiplication with every configuration, on either side, preserves that configuration. A candidate identity satisfies this requirement precisely when it preserves every primitive configuration on both sides.

In the general construction, the multiplicative identity may contain several primitive occurrences. Its existence depends on the composition table.

Whenever it exists, the multiplicative identity is unique. Its canonical identifier is determined by the enumeration and need not be identifier one.

When multiplication is associative and possesses a multiplicative identity, the configuration space forms a **unital semiring**: addition is a commutative monoid operation, multiplication is a monoid operation, multiplication distributes over addition, and the additive identity is multiplicatively absorbing.

When multiplication is also commutative, the structure is a **commutative unital semiring**.

Within a unital instance, ordinary nonnegative scalar replication has an internal representation. The scalar configuration associated with a count is the aggregate of that many copies of the multiplicative identity. Multiplying by that scalar configuration replicates the other operand by the specified count.

Division as exact multiplicative reconstruction

Object division is the collection of all admissible configurations that reconstruct a specified dividend through multiplication with a specified divisor.

When multiplication can be noncommutative, division requires an explicit side convention.

For **division with a right divisor**, a quotient is admissible precisely when multiplying the quotient by the divisor on its right reproduces the dividend.

For **division with a left divisor**, a quotient is admissible precisely when multiplying the divisor by the quotient on its right reproduces the dividend.

The output contains every quotient satisfying the selected reconstruction condition.

Division is therefore a total operation into the collection of subsets of the configuration space. Every operand pair determines a solution set, including operand pairs whose solution set is empty.

A division result can contain no configurations, exactly one configuration, several configurations, or infinitely many configurations.

The **empty solution set** means that no admissible quotient reconstructs the dividend. A **singleton solution set containing the empty configuration** means

that the empty configuration is the unique admissible quotient. These outcomes have different mathematical meanings.

Fixing a divisor turns multiplication into an additive mapping on the configuration space. Division returns a **fiber** of this mapping: all configurations mapped to the specified dividend.

In multiplicity coordinates, quotient recovery is consequently a system of linear constraints over nonnegative integers. The primitive composition table and the fixed divisor determine the coefficients of that system.

For a fixed divisor, every nonempty division fiber contains exactly one configuration precisely when multiplication by that divisor is injective on the relevant side. Associativity and the existence of an identity do not by themselves establish this injectivity.

The absorbing property of the empty configuration gives two universal cases. A nonempty dividend divided by an empty divisor has no quotient. An empty dividend divided by an empty divisor admits every configuration as a quotient.

With general primitive composition, an empty dividend divided by a nonempty divisor may admit nonempty quotients when the divisor annihilates those configurations.

Division with a remainder requires further specification, including the admissible residuals and the criterion selecting a quotient. Exact multiplicative reconstruction supplies the division relation defined here.

The finite monoid specialization

A particularly structured instance arises when every primitive pair produces exactly one primitive occurrence, primitive composition is associative, and the alphabet contains a primitive identity type.

Under these assumptions, the primitive alphabet forms a **finite monoid**.

Its configuration algebra is the **monoid semiring over the nonnegative integers**. Configurations are finite formal combinations of monoid elements with nonnegative integer coefficients, and multiplication is convolution induced by primitive monoid composition. The singleton configuration containing the primitive identity is the multiplicative identity. This is the standard monoid algebra construction specialized to nonnegative integer coefficients. [Mathlib, Monoid Algebra Definitions](#)

This specialization yields a stronger counting law. Every ordered pair of input occurrences contributes exactly one output occurrence. Consequently, the cardinality of a product is the product of the cardinalities of its operands.

Cardinality therefore preserves addition, multiplication, the additive identity, and the multiplicative identity. It is a **semiring homomorphism** from the configuration algebra to ordinary nonnegative integer arithmetic.

Several division properties follow directly from this counting law.

For a nonempty divisor, any quotient must have cardinality equal to the dividend cardinality divided by the divisor cardinality. If this requires a noninteger count, the quotient collection is empty.

Integer divisibility of cardinalities is necessary for exact division, but the primitive multiplicity distribution must also satisfy the composition constraints.

When the required quotient cardinality is an integer, only finitely many configurations have that cardinality. Every division problem with a nonempty divisor therefore has a finite quotient collection in this specialization.

An empty dividend divided by a nonempty divisor has exactly the empty configuration as its quotient.

For the general composition table, cardinality remains additive, while its multiplicative behavior depends on how many occurrences each primitive pair produces.

Semantic interpretation and algebraic soundness

A **semantic interpretation** maps configurations to elements of a declared application domain.

That domain might consist of resource states, inventories, formal transformation collections, production descriptions, or other explicitly specified semantic objects.

An interpreted instance must state which semantic operations correspond to configuration aggregation and composition. The interpretation is **operation preserving** when interpreting a configuration result agrees with applying the corresponding semantic operation to the interpreted operands.

For addition, interpreting an aggregate must agree with semantic aggregation of the operand meanings.

For multiplication, interpreting a product must agree with semantic composition of the operand meanings.

When these compatibility conditions hold, the interpretation is a homomorphism for the corresponding algebraic structure.

Inverse operations require an additional distinction between **soundness** and **completeness**.

Semantic soundness of subtraction means that interpreting an exact residual gives a semantic object which, when aggregated with the interpreted removal operand, reconstructs the interpreted original operand.

Semantic soundness of division means that interpreting any returned quotient gives a valid semantic factorization of the interpreted dividend.

Preservation of addition and multiplication establishes these reconstruction properties. It does not automatically establish that every semantic residual or quotient corresponds to an admissible configuration solution. An interpretation may discard distinctions that constrain exact configuration reconstruction.

Two configurations are **semantically equivalent** when their interpretations agree. They may remain distinct configurations with different canonical identifiers.

If semantic equivalence is used to merge configurations into equivalence classes, it must be a **congruence** for every operation retained on those classes. For addition and multiplication, replacing operands with equivalent configurations must preserve the equivalence class of the result.

Exact subtraction additionally requires compatible definedness. Division requires compatible quotient behavior. A congruence for addition and multiplication alone does not automatically preserve these inverse operation properties.

For example, an interpretation that retains only total cardinality identifies single occurrences of different primitive types. Exact subtraction still distinguishes their types: removing one type from an available occurrence of another type fails.

Admissibility restrictions and signed extensions

The preceding definition uses the complete space of finite multisets. An application may select a restricted domain through capacity limits, typing constraints, resource invariants, or other admissibility predicates.

Such a restriction must state its closure properties.

Addition remains total within the restricted domain only when aggregating any two admissible configurations yields another admissible configuration.

Multiplication remains total only when every product of admissible configurations is admissible.

Exact subtraction additionally requires the computed residual to belong to the restricted domain. Division collects only quotients that satisfy its admissibility conditions.

The free commutative monoid classification applies to the complete multiset construction. A restricted domain requires its own structural classification and a bijective identifier scheme covering exactly its admissible states.

A **signed extension** replaces nonnegative multiplicities with integer coefficients.

Its additive structure is the free abelian group generated by the primitive alphabet. Subtraction becomes total because negative coefficients are admitted.

Negative coefficients receive explicit semantics, such as deficits, obligations, or formal additive inverses. Their interpretation is part of the signed model.

Primitive composition extends through integer bilinearity. With associative multiplication and a multiplicative identity, this produces a ring. In the finite monoid specialization, it produces the corresponding **monoid ring over the integers**.

Signed configurations require an extended encoder and decoder. An enumeration by increasing sum of absolute coefficient values, followed by a fixed ordering within each finite level, provides one suitable sequential identification.

A fully specified illustrative instance

Consider an alphabet containing the primitive types **neutral** and **toggle**, with neutral declared first in the alphabet order.

Interpret neutral as a transformation that preserves a binary state, and toggle as a transformation that reverses it. Configurations record counted collections of these transformations. Multiplication constructs the collection of all prescribed pairwise composites.

Primitive composition follows four rules. Neutral composed with neutral produces neutral. Neutral composed with toggle produces toggle. Toggle composed with neutral produces toggle. Toggle composed with toggle produces neutral.

These rules form a finite commutative monoid.

Under increasing cardinality followed by ascending lexicographic multiplicity order, the empty configuration receives identifier zero. One toggle receives identifier one. One neutral receives identifier two. Two toggles receive identifier three. One neutral together with one toggle receives identifier four. Two neutrals receive identifier five.

Object addition applied to identifier one and identifier two returns identifier four, because aggregating one toggle with one neutral produces the mixed configuration.

Object subtraction applied to identifier four and identifier one returns identifier two, because removing one toggle from the mixed configuration leaves one neutral.

Object multiplication applied to identifier one and identifier one returns identifier two, because composing toggle with toggle produces neutral.

Object division of identifier four by identifier four returns the collection containing identifier one and identifier two.

The singleton neutral configuration is a quotient because it preserves the mixed configuration under multiplication. The singleton toggle configuration is also a quotient because it exchanges the neutral and toggle contributions, leaving the same multiplicity distribution.

These are the only quotients. The dividend and divisor each contain two occurrences, so every quotient must contain exactly one occurrence. The alphabet supplies exactly the two singleton candidates already identified.

This instance realizes addition, exact subtraction, associative multiplication, and nonunique division within one canonical identifier system. Its quotient multiplicity follows from the primitive composition rules and remains invariant under any consistent reindexing of the configurations.

prompt2

Formally define 'canonical indexing' in contrast to "sequential ID's".

Canonical indexing and sequential identifiers

Canonical indexing specifies the reproducible correspondence between a configuration and its address under a fixed mathematical convention.

Sequential identification specifies the succession of the labels used as addresses.

Canonicity therefore concerns the **determination of identity**, while sequentiality concerns the **progression of identifiers**. An indexing system can possess both properties.

Your engineering codex defines precisely this combination: **canonical sequential indexing**, obtained by ordering configurations by cardinality, resolving each cardinality level through lexicographic multiplicity order, and assigning consecutive identifiers beginning with zero. [Engineering codex](#)

Formal definition of canonical indexing

For Object Counting Algebra, **canonical indexing is an index assignment determined entirely by a fixed specification of the configuration domain, configuration equality, and indexing convention, such that every admissible configuration receives one unique index and configurations receive the same index precisely when they are equal.**

This definition contains several distinct requirements.

Totality requires every admissible configuration to have an index.

Uniqueness of assignment requires each configuration to have exactly one index within the specified indexing system.

Injectivity requires distinct configurations to have distinct indices.

Representation invariance requires equivalent presentations of the same configuration to receive the same index.

History independence requires the index to remain independent of incidental discovery order, registration order, storage order, and computational derivation.

Reversibility requires a decoder that recovers the configuration associated with each valid index.

Together, these requirements establish a bijection between the configuration domain and the domain of valid indices. Encoding followed by decoding recovers the same configuration. Decoding followed by encoding recovers the same valid index.

The domain of valid indices is part of the specification. General canonical indexing does not require every nonnegative integer to be a valid index, nor does it require consecutive numerical labels. Those are additional properties of a particular encoding convention.

For your algebra, the canonical assignment is **extensional**: its result depends on the configuration's primitive multiplicities. Different computational procedures that produce the same multiplicity distribution therefore produce the same canonical configuration index.

Formal definition of sequential identifiers

A **sequential identifier** is an identifier assigned according to a declared succession of labels.

In the strict consecutive numerical convention used by your codex, the label sequence begins with zero, followed by one, two, three, and each subsequent nonnegative integer.

Sequential indexing assigns these successive labels to successive members of a selected enumeration.

Its defining information is therefore an enumeration and a successor convention. The first enumerated entity receives the first label, the second receives the next label, and the assignment continues through the sequence.

Sequentiality alone does not determine which entity belongs at each position. The enumeration might follow registration order, discovery order, cardinality order, lexicographic order, or another declared ordering.

Consequently, sequential identifiers can be assigned reproducibly from mathematical structure, or they can depend on operational history.

A registry might assign the next available identifier whenever it encounters a previously unregistered configuration. Such a registry can maintain uniqueness within its own records while assigning different identifiers from another registry that encounters the same configurations in a different order.

Both registries can be sequential. Their configuration assignments differ because their enumeration histories differ.

Chronology is therefore one possible basis for sequential identification. Sequentiality itself does not require chronological assignment.

The formal contrast

The distinction can be expressed through two separate questions.

Canonical indexing determines which index belongs to a particular configuration under the fixed specification.

Sequential identification determines which label follows another label in the selected enumeration.

Canonical indexing constrains the correspondence between configuration identity and index identity. Sequential identification constrains the progression of labels through an enumeration.

A canonical encoding could assign a normalized multiplicity description as the configuration's index. That description could be unique and reproducible without being a consecutive integer.

Conversely, a registry could assign consecutive integers according to arrival order. Its labels would be sequential, while their association with configurations would depend on arrival history.

A **canonical sequential index** satisfies both requirements. A fixed ordering of configurations determines each configuration's position, and that position supplies its consecutive numerical identifier.

Uniqueness alone is insufficient to establish canonicity. Many different bijections can assign unique integers to the same configuration domain. Canonicity selects a distinguished correspondence through an explicitly fixed convention.

Determinism alone is also insufficient. An allocator may deterministically assign the next identifier to each arriving configuration, yet produce a different correspondence when the arrival order changes. For canonical indexing, the relevant reproducibility is reproducibility from the configuration and the fixed specification.

Canonical representation and canonical index

A **canonical representation** is the normalized description of a configuration.

A **canonical index** is the address assigned to that configuration under the indexing convention.

In your multiset model, the canonical representation is the multiplicity distribution written in the fixed alphabet order. If the alphabet contains neutral followed by toggle, the representation records the neutral multiplicity first and the toggle multiplicity second.

A presentation containing one neutral followed by one toggle describes the same configuration as a presentation containing one toggle followed by one neutral. Both normalize to the same multiplicity distribution and receive the same canonical index.

The normalization identifies configuration equality. The indexing convention assigns the address.

A system may compute these as separate stages, although a direct encoder can combine them into one procedure.

The decoder recovers the indexed configuration or its canonical representation. It need not reconstruct incidental details of the original presentation, such as the order in which interchangeable occurrences were written.

This distinction also clarifies the object being identified. Repeated presentations of one multiset receive the same canonical configuration index. Individually distinguishable occurrences, stored records, or evaluation events require their own identity domain if they are to receive separate identifiers.

Canonical sequential indexing in Object Counting Algebra

Your codex fixes the following mathematical construction.

First, declare a finite, nonempty, ordered alphabet of primitive object types.

Next, represent every configuration by its nonnegative multiplicities in that alphabet order.

Order the complete configuration domain by increasing total cardinality. Within each cardinality level, order configurations by ascending lexicographic comparison of their multiplicities.

Finally, assign consecutive identifiers beginning with zero.

The resulting **canonical index is the configuration's rank in this declared enumeration.**

With indexing beginning at zero, the rank is the number of configurations preceding the indexed configuration. It comprises every configuration with smaller cardinality, together with every configuration of equal cardinality that precedes it lexicographically.

This definition gives each configuration a finite rank because the alphabet is finite and every cardinality level is finite.

That finiteness condition is essential. A total order on an infinite domain does not automatically supply a consecutive natural number ranking.

For example, unrestricted ascending lexicographic order on the neutral and toggle multiplicities would place every configuration containing zero neutrals before a configuration containing one neutral. There are infinitely many configurations containing zero neutrals because the toggle multiplicity is unbounded. The configuration containing one neutral would therefore have infinitely many predecessors.

Ordering by cardinality before applying lexicographic comparison prevents this obstruction. Every configuration has only finitely many predecessors and therefore receives a finite sequential index.

The encoder is consequently a **ranking operation**, and the decoder is an **unranking operation**.

A direct comparison using neutral and toggle

Take the primitive alphabet from your codex, with neutral ordered before toggle.

Under its canonical enumeration, the empty configuration receives identifier zero. One toggle receives identifier one. One neutral receives identifier two. Two toggles receive identifier three. One neutral together with one toggle receives identifier four. Two neutrals receive identifier five.

These assignments follow from the fixed configuration ordering.

Now consider a separate registry that reserves identifier zero for the empty configuration and assigns each subsequent identifier according to arrival order.

Suppose the mixed configuration containing one neutral and one toggle arrives first. The registry assigns it identifier one. If one toggle arrives next, that configuration receives identifier two. If one neutral arrives afterward, it receives identifier three.

This registry uses sequential identifiers. Its numbering differs from the canonical enumeration because the assignment follows arrival order.

Under your canonical indexing convention, the mixed configuration receives identifier four even when it is the first nonempty configuration encountered.

Its index is determined by its position in the complete mathematical configuration domain. No prior registration of the configurations at identifiers one, two, and three is required.

This produces a useful distinction between the **complete indexed domain** and the **currently stored configurations**. The complete domain can have consecutive canonical indices while the configurations currently stored occupy only selected positions within that domain.

Canonicity is relative to a specification

A canonical index is canonical relative to the structure and conventions that determine it.

For your algebra, these include the primitive alphabet, the alphabet order, the configuration equality criterion, the admissible configuration domain, the enumeration rule, and the starting index.

Different conventions can produce different canonical assignments.

Reversing the alphabet order can change lexicographic ranks. Expanding the alphabet can introduce additional configurations into earlier cardinality levels. Replacing nonnegative multiplicities with signed coefficients requires a different enumeration. Restricting the admissible domain can change which configurations precede a given configuration.

Each resulting scheme may be internally canonical, provided its specification determines the assignment consistently.

Accordingly, an identifier is fully interpretable within its **indexing namespace**: the particular configuration model and indexing convention to which it belongs. Equality of numerical labels across different namespaces does not establish equality of the referenced configurations.

Canonicity also has a precise stability boundary.

Discovering another configuration within an already fixed complete domain leaves existing canonical indices unchanged. The newly encountered configuration already has its mathematically determined rank.

Changing the domain or ordering convention may change existing ranks. A system that ranks only its currently stored configurations can likewise require reindexing when a newly inserted configuration precedes existing entries.

The codex's construction avoids dependence on the currently stored subset by indexing the complete declared configuration domain.

Configuration identity, semantic equivalence, and arithmetic

Canonical indexing must use an explicit equality criterion.

In the basic Object Counting Algebra, two configurations are equal precisely when their primitive multiplicities agree. Their canonical indices therefore agree precisely under that condition.

Two different configurations may nevertheless have the same application interpretation. Such semantic equivalence does not automatically merge their canonical indices.

If the intended indexed entities are semantic equivalence classes, the equivalence relation must become part of the indexing specification. Encoding then identifies classes, and decoding recovers a class or a selected

representative. Using those classes as algebraic states additionally requires the relevant operations to respect the equivalence relation.

Canonical indexing also leaves the interpretation of arithmetic explicit.

Identifier one denotes the singleton toggle configuration, and identifier two denotes the singleton neutral configuration. Their object addition returns identifier four because their aggregate is the mixed configuration.

The consecutive numerical ordering of the addresses does not determine that operation. The operation is defined through decoding, configuration aggregation, and reencoding.

The precise designation for the scheme in your codex is therefore **canonical sequential configuration indexing**: configuration equality determines the indexed states, the declared ordering determines their ranks, and consecutive identifiers express those ranks.

Object Counting Algebra

The following is a single parameterized specification of the multiset construction in your engineering codex. The general algebra, canonical encoding, operational semantics, and conditional specializations are incorporated into one formulation.

$$\mathfrak{D}(A, T, \mathcal{S}, \sigma) := \left\langle \mathcal{C}, \mathbf{0}, \nu, \oplus, \star, \ominus, Q_R, Q_L; \right. \\ \left. I, \rho, \delta, \hat{\oplus}, \hat{\star}, \hat{\ominus}, \hat{Q}_R, \hat{Q}_L; \right. \\ \left. \mathcal{S}, \sigma, \sigma_I, \text{Out}_I, \text{Eval} \right\rangle.$$

Primitive domain

$$\mathbb{N}_0 := \{0, 1, 2, \dots\}, \quad d \in \mathbb{N}_{\geq 1}, \quad [d] := \{1, \dots, d\},$$

$$A := \{o_1, \dots, o_d\}, \quad o_1 <_A o_2 <_A \dots <_A o_d,$$

$$\mathcal{C} := \left\{ \sum_{i=1}^d x_i o_i : x_i \in \mathbb{N}_0 \right\} \cong \mathbb{N}_0^d,$$

$$x = (x_1, \dots, x_d) \in \mathcal{C}, \quad (e_i)_k := \mathbf{1}_{\{i=k\}},$$

$$x = \bigoplus_{i=1}^d x_i e_i, \quad \mathbf{0} := (0, \dots, 0),$$

$$x = y \iff \forall i \in [d], x_i = y_i,$$

$$x \preceq y \iff \forall i \in [d], x_i \leq y_i,$$

$$\nu : \mathcal{C} \longrightarrow \mathbb{N}_0, \quad \nu(x) := \sum_{i=1}^d x_i,$$

$$\mathcal{C}_n := \{x \in \mathcal{C} : \nu(x) = n\}, \quad |\mathcal{C}_n| = \binom{n+d-1}{d-1},$$

$$\mathcal{C} = \bigsqcup_{n \in \mathbb{N}_0} \mathcal{C}_n, \quad |\mathcal{C}| = \aleph_0.$$

Canonical sequential indexing

$$x <_{\text{lex}} y \iff \exists r \in [d] : (x_r < y_r \wedge \forall h < r, x_h = y_h),$$

$$x \prec y \iff \nu(x) < \nu(y)$$

$$\vee (\nu(x) = \nu(y) \wedge x <_{\text{lex}} y),$$

$$I := \mathbb{N}_0,$$

$$\rho : \mathcal{C} \longrightarrow I, \quad \rho(x) := |\{y \in \mathcal{C} : y \prec x\}|,$$

$$p_i(x) := \sum_{h=1}^{i-1} x_h,$$

$$\rho(x) = \binom{\nu(x) + d - 1}{d}$$

$$+ \sum_{i=1}^{d-1} \sum_{a=0}^{x_i-1} \binom{\nu(x) - p_i(x) - a + d - i - 1}{d - i - 1},$$

$$\sum_{\emptyset} := 0, \quad \binom{n}{k} := 0 \quad (0 \leq n < k),$$

$$\forall a \in I, \exists! x \in \mathcal{C} : \rho(x) = a,$$

$$\delta := \rho^{-1} : I \longrightarrow \mathcal{C},$$

$$\delta \circ \rho = \text{id}_{\mathcal{C}}, \quad \rho \circ \delta = \text{id}_I,$$

$$\rho(\mathbf{0}) = 0, \quad \rho(e_i) = d - i + 1,$$

$$\rho(x) = \rho(y) \iff x = y.$$

Primitive composition and configuration operations

$$T = (T_{ij}^k)_{i,j,k \in [d]} \in \mathbb{N}_0^{d \times d \times d},$$

$$\mu_T : A \times A \longrightarrow \mathcal{C}, \quad \mu_T(o_i, o_j) := \sum_{k=1}^d T_{ij}^k e_k,$$

$$\oplus, \star : \mathcal{C} \times \mathcal{C} \longrightarrow \mathcal{C},$$

$$(x \oplus y)_k := x_k + y_k,$$

$$(x \star y)_k := \sum_{i=1}^d \sum_{j=1}^d x_i y_j T_{ij}^k,$$

$$e_i \star e_j = \mu_T(o_i, o_j),$$

$$\ominus : \mathcal{C} \times \mathcal{C} \rightarrow \mathcal{C},$$

$$\text{dom}(\ominus) := \{(x, y) \in \mathcal{C}^2 : y \preceq x\},$$

$$(x \ominus y)_k := x_k - y_k \quad ((x, y) \in \text{dom}(\ominus)),$$

$$\mathcal{R}(x, y) := \{z \in \mathcal{C} : z \oplus y = x\}$$

$$= \begin{cases} \{x \ominus y\}, & y \preceq x, \\ \emptyset, & y \not\preceq x, \end{cases}$$

$$Q_R, Q_L : \mathcal{C} \times \mathcal{C} \rightarrow \mathcal{P}(\mathcal{C}),$$

$$Q_R(x, y) := \{q \in \mathcal{C} : q \star y = x\},$$

$$Q_L(x, y) := \{q \in \mathcal{C} : y \star q = x\}.$$

Unconditional algebraic laws

$$\forall x, y, z \in \mathcal{C} :$$

$$(x \oplus y) \oplus z = x \oplus (y \oplus z),$$

$$x \oplus y = y \oplus x, \quad x \oplus \mathbf{0} = x,$$

$$x \oplus z = y \oplus z \iff x = y,$$

$$x \oplus y = \mathbf{0} \iff x = y = \mathbf{0},$$

$$x \star (y \oplus z) = (x \star y) \oplus (x \star z),$$

$$(x \oplus y) \star z = (x \star z) \oplus (y \star z),$$

$$x \star \mathbf{0} = \mathbf{0} \star x = \mathbf{0},$$

$$\nu(x \oplus y) = \nu(x) + \nu(y),$$

$$y \preceq x \implies (x \ominus y) \oplus y = x,$$

$$(x \oplus y) \ominus y = x, \quad x \ominus x = \mathbf{0},$$

$$\begin{aligned}
(\mathcal{C}, \oplus, \mathbf{0}) &\cong \text{FreeCMon}(A), \\
\nu(x \star y) &= \sum_{i=1}^d \sum_{j=1}^d x_i y_j \left(\sum_{k=1}^d T_{ij}^k \right), \\
[\forall x, y \in \mathcal{C}, \nu(x \star y) &= \nu(x)\nu(y)] \\
&\iff \left[\forall i, j \in [d], \sum_{k=1}^d T_{ij}^k = 1 \right].
\end{aligned}$$

Associativity, commutativity, and multiplicative identity

$$\begin{aligned}
A_T &\iff \forall i, j, k, m \in [d] : \\
&\sum_{\ell=1}^d T_{ij}^{\ell} T_{\ell k}^m = \sum_{\ell=1}^d T_{jk}^{\ell} T_{i\ell}^m, \\
C_T &\iff \forall i, j, k \in [d], T_{ij}^k = T_{ji}^k, \\
\mathcal{U}_T &:= \left\{ u \in \mathcal{C} : \begin{array}{l} \forall i, k \in [d], \\ \sum_{j=1}^d u_j T_{ji}^k = \mathbf{1}_{\{i=k\}} = \sum_{j=1}^d u_j T_{ij}^k \end{array} \right\}, \\
A_T &\iff \forall x, y, z \in \mathcal{C}, (x \star y) \star z = x \star (y \star z), \\
C_T &\iff \forall x, y \in \mathcal{C}, x \star y = y \star x, \\
|\mathcal{U}_T| &\leq 1, \\
u \in \mathcal{U}_T &\implies u \neq \mathbf{0} \wedge \forall x \in \mathcal{C}, u \star x = x \star u = x, \\
A_T \wedge u \in \mathcal{U}_T &\implies (\mathcal{C}, \oplus, \star, \mathbf{0}, u) \in \text{Semiring}_1, \\
A_T \wedge C_T \wedge u \in \mathcal{U}_T &\implies (\mathcal{C}, \oplus, \star, \mathbf{0}, u) \in \text{CSemiring}_1, \\
u \in \mathcal{U}_T &\implies \forall n \in \mathbb{N}_0, \forall x \in \mathcal{C} : \\
&(nu) \star x = x \star (nu) = nx.
\end{aligned}$$

Division as nonnegative integer fibers

$$(M_{\mathbf{R}}(y))_{ki} := \sum_{j=1}^d T_{ij}^k y_j,$$

$$(M_{\mathbf{L}}(y))_{ki} := \sum_{j=1}^d T_{ji}^k y_j,$$

$$q \star y = M_{\mathbf{R}}(y)q, \quad y \star q = M_{\mathbf{L}}(y)q,$$

$$\varepsilon \in \{\mathbf{R}, \mathbf{L}\},$$

$$Q_{\varepsilon}(x, y) = \{q \in \mathbb{N}_0^d : M_{\varepsilon}(y)q = x\},$$

$$Q_{\varepsilon}(x, \mathbf{0}) = \begin{cases} \mathcal{C}, & x = \mathbf{0}, \\ \emptyset, & x \neq \mathbf{0}, \end{cases}$$

$$Q_{\varepsilon}(\mathbf{0}, y) = \{q \in \mathbb{N}_0^d : M_{\varepsilon}(y)q = \mathbf{0}\},$$

$$c_i^{\varepsilon}(y) := \sum_{k=1}^d (M_{\varepsilon}(y))_{ki},$$

$$q \in Q_{\varepsilon}(x, y) \implies \sum_{i=1}^d c_i^{\varepsilon}(y)q_i = \nu(x),$$

$$|Q_{\varepsilon}(x, y)| = \aleph_0$$

$$\iff Q_{\varepsilon}(x, y) \neq \emptyset \wedge \exists i \in [d], c_i^{\varepsilon}(y) = 0,$$

$$[\forall x \in \mathcal{C}, |Q_{\varepsilon}(x, y)| \leq 1] \iff \det M_{\varepsilon}(y) \neq 0.$$

Transport to identifier arithmetic

$$\hat{\oplus}, \hat{\star} : I^2 \longrightarrow I, \quad \hat{\ominus} : I^2 \rightharpoonup I,$$

$$\hat{Q}_{\mathbf{R}}, \hat{Q}_{\mathbf{L}} : I^2 \longrightarrow \mathcal{P}(I),$$

$$a \hat{\oplus} b := \rho(\delta(a) \oplus \delta(b)),$$

$$a \hat{\star} b := \rho(\delta(a) \star \delta(b)),$$

$$\begin{aligned}
\text{dom}(\widehat{\ominus}) &= \{(a, b) \in I^2 : \delta(b) \preceq \delta(a)\}, \\
a\widehat{\ominus}b &:= \rho(\delta(a) \ominus \delta(b)) \quad ((a, b) \in \text{dom}(\widehat{\ominus})), \\
\rho[U] &:= \{\rho(x) : x \in U\} \quad (U \subseteq \mathcal{C}), \\
\widehat{Q}_\varepsilon(a, b) &:= \rho[Q_\varepsilon(\delta(a), \delta(b))], \\
0_I &:= \rho(\mathbf{0}) = 0, \quad u \in \mathcal{U}_T \implies 1_I := \rho(u), \\
\nu_I &:= \nu \circ \delta, \\
\nu_I(a\widehat{\oplus}b) &= \nu_I(a) + \nu_I(b).
\end{aligned}$$

Typed outcomes and total evaluation

$$\begin{aligned}
\text{Out}_I &:= \text{Val}(I) \sqcup \text{Sol}(\mathcal{P}(I)) \sqcup \{\text{Fail}\}, \\
|\{\text{Val}(0), \text{Sol}(\emptyset), \text{Sol}(\{0\}), \text{Fail}\}| &= 4, \\
\Omega_{\text{alg}} &:= \{\oplus, \star, \ominus\}, \\
\Omega &:= \Omega_{\text{alg}} \sqcup \{\text{div}_R, \text{div}_L\}, \\
\text{Eval} : \Omega \times I \times I &\longrightarrow \text{Out}_I, \\
\text{Eval}(\omega, a, b) &:= \begin{cases} \text{Val}(a\widehat{\omega}b), & \omega \in \Omega_{\text{alg}}, \\ & (a, b) \in \text{dom}(\widehat{\omega}), \\ \text{Fail}, & \omega \in \Omega_{\text{alg}}, \\ & (a, b) \notin \text{dom}(\widehat{\omega}), \\ \text{Sol}(\widehat{Q}_\varepsilon(a, b)), & \omega = \text{div}_\varepsilon. \end{cases}
\end{aligned}$$

Semantic interpretation and compatibility

$$\begin{aligned}
\mathcal{S} &:= (S, \oplus_S, \star_S, 0_S), \\
\oplus_S, \star_S : S^2 &\longrightarrow S, \quad 0_S \in S, \\
\sigma : \mathcal{C} &\longrightarrow S, \quad \sigma_I := \sigma \circ \delta,
\end{aligned}$$

$$H_\sigma \iff \begin{cases} \sigma(\mathbf{0}) = 0_S, \\ \forall x, y \in \mathcal{C} : \\ \sigma(x \oplus y) = \sigma(x) \oplus_S \sigma(y), \\ \sigma(x \star y) = \sigma(x) \star_S \sigma(y), \end{cases}$$

$$\mathcal{R}_S(s, t) := \{v \in S : v \oplus_S t = s\},$$

$$Q_R^S(s, t) := \{v \in S : v \star_S t = s\},$$

$$Q_L^S(s, t) := \{v \in S : t \star_S v = s\},$$

$$\sigma[U] := \{\sigma(x) : x \in U\},$$

$$H_\sigma \implies \sigma[\mathcal{R}(x, y)] \subseteq \mathcal{R}_S(\sigma(x), \sigma(y)),$$

$$H_\sigma \implies \sigma[Q_R(x, y)] \subseteq Q_R^S(\sigma(x), \sigma(y)),$$

$$H_\sigma \implies \sigma[Q_L(x, y)] \subseteq Q_L^S(\sigma(x), \sigma(y)),$$

$$x \sim_\sigma y \iff \sigma(x) = \sigma(y),$$

$$H_\sigma \implies \left[\begin{array}{cc} x \sim_\sigma x', & y \sim_\sigma y' \\ \Downarrow & \\ x \oplus y \sim_\sigma x' \oplus y', & \\ x \star y \sim_\sigma x' \star y' & \end{array} \right],$$

$$\left[\begin{array}{l} \forall x, x', y, y' \in \mathcal{C} : \\ x \sim_\sigma x' \wedge y \sim_\sigma y' \\ \implies (y \preceq x \iff y' \preceq x') \end{array} \right] \\ \implies [x \sim_\sigma y \iff x = y].$$

Reindexing invariance

$$\pi : I \xrightarrow{\sim} I, \quad \rho_\pi := \pi \circ \rho, \quad \delta_\pi := \delta \circ \pi^{-1},$$

$$\widehat{\omega}_\pi := \pi \circ \widehat{\omega} \circ (\pi^{-1} \times \pi^{-1}) \quad (\omega \in \Omega_{\text{alg}}),$$

$$\widehat{Q}_{\varepsilon, \pi}(a, b) := \pi \left[\widehat{Q}_\varepsilon(\pi^{-1}(a), \pi^{-1}(b)) \right],$$

$$\sigma_{I, \pi} := \sigma_I \circ \pi^{-1},$$

$$0_{I,\pi} = \pi(0_I), \quad u \in \mathcal{U}_T \implies 1_{I,\pi} = \pi(1_I),$$

$$\mathfrak{D}_I \cong_\pi \mathfrak{D}_{I,\pi}.$$

Finite monoid specialization

$$m : [d] \times [d] \longrightarrow [d], \quad h \in [d],$$

$$\mathbf{M}_{m,h}(T) \iff \begin{cases} T_{ij}^k = \mathbf{1}_{\{m(i,j)=k\}} & \forall i, j, k \in [d], \\ m(m(i, j), k) = m(i, m(j, k)) & \forall i, j, k \in [d], \\ m(h, i) = m(i, h) = i & \forall i \in [d], \end{cases}$$

$$\mathbf{M}_{m,h}(T) \implies M := ([d], m, h) \in \mathbf{Mon},$$

$$\mathbf{M}_{m,h}(T) \implies (\mathcal{C}, \oplus, \star, \mathbf{0}, e_h) \cong \mathbb{N}_0[M],$$

$$\mathbf{M}_{m,h}(T) \implies \mathcal{U}_T = \{e_h\}, \quad 1_I = \rho(e_h) = d - h + 1,$$

$$\mathbf{M}_{m,h}(T) \implies \nu(x \star y) = \nu(x)\nu(y),$$

$$\mathbf{M}_{m,h}(T) \implies \nu : \mathbb{N}_0[M] \longrightarrow \mathbb{N}_0 \text{ is a unital semiring homomorphism,}$$

$$\mathbf{M}_{m,h}(T) \implies \nu_I(a \hat{\star} b) = \nu_I(a)\nu_I(b),$$

$$\mathbf{M}_{m,h}(T) \wedge y \neq \mathbf{0}$$

$$\implies Q_\varepsilon(x, y) \subseteq \{q \in \mathcal{C} : \nu(q)\nu(y) = \nu(x)\},$$

$$\mathbf{M}_{m,h}(T) \wedge y \neq \mathbf{0} \wedge \nu(y) \nmid \nu(x)$$

$$\implies Q_\varepsilon(x, y) = \emptyset,$$

$$\mathbf{M}_{m,h}(T) \wedge y \neq \mathbf{0} \wedge N := \frac{\nu(x)}{\nu(y)} \in \mathbb{N}_0$$

$$\implies |Q_\varepsilon(x, y)| \leq \binom{N + d - 1}{d - 1} < \infty,$$

$$\mathbf{M}_{m,h}(T) \wedge y \neq \mathbf{0} \implies Q_\varepsilon(\mathbf{0}, y) = \{\mathbf{0}\}.$$

Admissibility restriction

$$\mathbf{0} \in D \subseteq \mathcal{C}, \quad \omega \in \{\oplus, \star, \ominus\},$$

$$\text{dom}(\omega_D) := \left\{ (x, y) \in D^2 : \begin{array}{l} (x, y) \in \text{dom}(\omega), \\ \omega(x, y) \in D \end{array} \right\},$$

$$\omega_D(x, y) := \omega(x, y) \quad ((x, y) \in \text{dom}(\omega_D)),$$

$$Q_{\varepsilon, D}(x, y) := Q_{\varepsilon}(x, y) \cap D, \quad (x, y) \in D^2,$$

$$\rho_D(x) := |\{y \in D : y \prec x\}|,$$

$$I_D := \rho_D[D] = \begin{cases} \{0, \dots, |D| - 1\}, & |D| < \infty, \\ \mathbb{N}_0, & |D| = \aleph_0, \end{cases}$$

$$\delta_D := \rho_D^{-1}, \quad \sigma_D := \sigma|_D,$$

$$\text{dom}(\oplus_D) = D^2 \iff \oplus[D^2] \subseteq D,$$

$$\text{dom}(\star_D) = D^2 \iff \star[D^2] \subseteq D,$$

$$\hat{\omega}_D := \rho_D \circ \omega_D \circ (\delta_D \times \delta_D),$$

$$\hat{Q}_{\varepsilon, D}(a, b) := \rho_D[Q_{\varepsilon, D}(\delta_D(a), \delta_D(b))].$$

Signed extension

$$\mathcal{C}^{\pm} := \mathbb{Z}^d \cong \text{Gr}(\mathcal{C}, \oplus),$$

$$(x, y) \sim_{\text{Gr}} (x', y') \iff x \oplus y' = x' \oplus y,$$

$$(\mathcal{C} \times \mathcal{C}) / \sim_{\text{Gr}} \longrightarrow \mathbb{Z}^d, \quad [(x, y)] \longmapsto x - y,$$

$$(z \oplus_{\pm} w)_k := z_k + w_k, \quad (z \ominus_{\pm} w)_k := z_k - w_k,$$

$$(z \star_{\pm} w)_k := \sum_{i=1}^d \sum_{j=1}^d z_i w_j T_{ij}^k,$$

$$\ominus_{\pm} : (\mathcal{C}^{\pm})^2 \longrightarrow \mathcal{C}^{\pm},$$

$$Q_{\text{R}}^{\pm}(z, w) := \{q \in \mathbb{Z}^d : q \star_{\pm} w = z\},$$

$$Q_L^\pm(z, w) := \{q \in \mathbb{Z}^d : w \star_\pm q = z\},$$

$$\|z\|_1 := \sum_{i=1}^d |z_i|,$$

$$z \prec_\pm w \iff \|z\|_1 < \|w\|_1$$

$$\vee (\|z\|_1 = \|w\|_1 \wedge z <_{\text{lex}} w),$$

$$\rho_\pm(z) := |\{w \in \mathbb{Z}^d : w \prec_\pm z\}|,$$

$$\rho_\pm : \mathbb{Z}^d \xrightarrow{\sim} \mathbb{N}_0, \quad \delta_\pm := \rho_\pm^{-1},$$

$$A_T \wedge u \in \mathcal{U}_T$$

$$\implies (\mathbb{Z}^d, \oplus_\pm, \star_\pm, \mathbf{0}, u) \in \text{Ring}_1,$$

$$M_{m,h}(T)$$

$$\implies (\mathbb{Z}^d, \oplus_\pm, \star_\pm, \mathbf{0}, e_h) \cong \mathbb{Z}[M].$$

Neutral-toggle instance

$$d = 2, \quad o_1 = \text{neutral}, \quad o_2 = \text{toggle},$$

$$T_{11}^1 = T_{12}^2 = T_{21}^2 = T_{22}^1 = 1,$$

$$T_{ij}^k = 0 \quad \text{for all remaining } (i, j, k),$$

$$(p, q) \star (r, s) = (pr + qs, ps + qr),$$

$$\rho(p, q) = \binom{p+q+1}{2} + p,$$

a	0	1	2	3	4	5
$\delta(a)$	(0, 0)	(0, 1)	(1, 0)	(0, 2)	(1, 1)	(2, 0)

$$1 \hat{\oplus} 2 = 4, \quad 4 \hat{\ominus} 1 = 2, \quad 1 \hat{\star} 1 = 2,$$

$$\hat{Q}_R(4, 4) = \hat{Q}_L(4, 4) = \{1, 2\},$$

$$Q_R((1, 1), (1, 1)) = \{(0, 1), (1, 0)\},$$

$$Q_R^\pm((1, 1), (1, 1)) = \{(t, 1-t) : t \in \mathbb{Z}\}.$$